



Introdução à Ciência da Computação

C++

Aprenda as sintaxes básicas, os loops while, e o ambiente do CodeCombat.

Índice

1. Masmorra Kithgard
2. Gemas nas Profundezas
3. Guarda Sombrio
- 3a. Kounter Kithwise
- 3b. Oculto em Kithgard
4. Desafio de Conceito. Passos Cuidadosos
5. Mina Inimiga
- 5a. Distração Ilusória
- 5b. Gemas Esquecidas
6. Desafio de Conceito. Passos Longos
7. Nomes Próprios
- 7a. Chances Favoráveis
- 7b. A Espada Erguida
8. Desafio de Conceito. Passos Perigosos
9. Desafio Combo. Hora de Dormir
10. Comentários na Cela
11. Bibliotecário de Kithgard
12. O Prisioneiro
13. Dança no Fogo
14. Labirinto Kithmaze
15. Continue a Descer
- 15a. Saindo de Kithmaze
- 15b. Pedra da Luz
16. Desafio de Conceito. Loop no Armazém

- 17. Porta Misteriosa
- 18. Bata e Corra
- 19. Armários de Kithgard
- 19a. Armários de Kithgard A
- 19b. Armários de Kithgard B
- 20. Inimigo Conhecido
- 21. Mestre dos Nomes
- 21a. Ataque Duplo
- 21b. Curta Distância
- 22. Desafio de Conceito. Mestre Dos Nomes de Depurar
- 23. O Labirinto Final
- 24. Desafio Combo. Lugar Seguro
- 25. Portões de Kithgard
- 26. Wakka Maul

#1. Masmorra Kithgard

Análise do Nível e Soluções

Colete a gema e escape da masmorra, mas não toque em nada mais. Neste nível, você aprenderá a movimentação básica do seu herói.

Introdução



Guie o seu herói escrevendo um programa!

Escreva o código no editor à direita e clique em Rodar. O seu herói irá ler e seguir as instruções.

Mova seu herói pelo corredor sem tocar nos espinhos das paredes.

Código padrão

```
// Mova-se até a gema.  
// Não toque nos espinhos!  
// Digite seu código abaixo e clique em Rodar.  
  
int main() {  
    hero.moveRight();  
  
    return 0;  
}
```

Visão Geral

A jornada começa! Para escapar da Masmorra Kithgard, o seu herói deve se mover. Você pode dizer a ele como mover-se escrevendo um *código*.

Escreva o seu código no editor para enviar as instruções. Os heróis leem e executam essas instruções sozinhos quando mandados. Para direcionar seu herói, refira-se a ele com o código `hero`.

Agora, você pode instruí-lo a mover-se com os comandos `moveDown` (mover para baixo) e `moveRight` (mover para a direita):

```
hero.moveDown();  
hero.moveRight();
```

Para ter sucesso nesse nível: mova-se para **direita**, **baixo**, e **direita** e pegue a gema! Você precisa de apenas *três linhas de código*.

O código que você escreveu aqui é muito similar ao código que você escreveria para dizer ao computador como fazer todos os tipos de coisas, de tocar uma música a exibir uma página da web. Você está dando os primeiros passos para se tornar um programador!

Masmorra Kithgard Solução

```
// Mova-se até a gema.  
// Não toque nos espinhos!  
// Digite seu código abaixo e clique em Rodar.  
  
int main() {  
    hero.moveRight();  
    hero.moveDown();  
    hero.moveRight();  
    return 0;  
}
```


#2. Gemas nas Profundezas

Análise do Nível e Soluções

Colete as gemas rapidamente. Você precisará delas.

Introdução



Lembre-se que é importante se mover:

```
hero.moveRight();
```

Código padrão

```
// Pegue todas as gemas usando seus comandos de movimento.  
  
int main() {  
    hero.moveRight();  
  
    return 0;  
}
```

Visão Geral

Você lembrar das lições do último nível? Neste você fará a mesma coisa, porém, terá que mover-se mais. Atenção, `hero` refere-se a você, o herói.

Quando você usa um comando `move`, ele te leva até o fim da área do movimento (olhe para os pequenos ladrilhos no chão).

Talvez você precise usar o `moveUp` (mover para cima) duas vezes para chegar ao topo desse nível.

Para evitar a repetição dos comandos podemos inserir, neste caso, um número dentro dos parênteses para indicar a quantidade de repetições do comando, o que na programação é chamado de **argumento**.

Por exemplo, para mover o herói para cima duas vezes você pode usar:

```
hero.moveUp(2);
```

Gemas nas Profundezas Solução

```
// Pegue todas as gemas usando seus comandos de movimento.
```

```
int main() {  
    hero.moveRight();  
    hero.moveDown();  
    hero.moveUp();  
    hero.moveUp();  
    hero.moveRight();  
    return 0;  
}
```

#3. Guarda Sombrio

Análise do Nível e Soluções

Escape do ogro para pegar as gemas e chegar do outro lado com segurança. Cuidado com os espinhos!

Introdução



Tente passar despercebido.

Código padrão

```
int main() {  
    // Fique fora da visão do ogro. Colete todas as gemas.  
    hero.moveRight();  
  
    return 0;  
}
```

Visão Geral

Como você não tem nenhuma arma ainda, não pode combater o Ogro Munchkin que protege o caminho.

Ao invés de enfrentá-lo, tente passar atrás da estátua para que ele não te veja. Assim você pode alcançar a gema e correr até o final sem ser detectado.

Guarda Sombrio Solução

```
int main() {  
    // Fique fora da visão do ogro. Colete todas as gemas.  
    hero.moveRight();  
    hero.moveUp();  
    hero.moveRight();  
    hero.moveDown();  
    hero.moveRight();  
    return 0;  
}
```

#3a. Kounter Kithwise

Análise do Nível e Soluções

Fique longe da linha de visão do Ogro Patrulheiro.

Introdução

Evite os Ogros Patrulheiros escolhendo seu caminho com cuidado!

Código padrão

```
int main() {  
    // Evite os ogros e pegue a gema.  
  
    return 0;  
}
```

Visão Geral

As vezes é tudo uma questão de tempo. Estes ogros possuem uma rota de patrulha. Você deve passar por ela sem ser visto para alcançar a gema.

Você pode arrastar a linha de tempo da reprodução para ver o que acontece na última vez que você rodou o código.

Kounter Kithwise Solução

```
// Evite os ogros e pegue a gema.  
  
int main() {  
    hero.moveDown(2);  
    hero.moveRight();  
    hero.moveUp();  
    hero.moveRight();  
    return 0;  
}
```

#3b. Oculto em Kithgard

Análise do Nível e Soluções

Dois corredores, uma solução. O tempo é essencial.

Introdução

Calcule seu tempo com cuidado para evitar a patrulha!

Código padrão

```
// Evite ser visto pelos ogros.  
  
int main() {  
    return 0;  
}
```

Visão Geral

Você deve usar sua inteligência para resolver o enigma. Movimente-se pela mmasmorra sem ser visto pelos Ogros Patrulheiros.

Oculto em Kithgard Solução

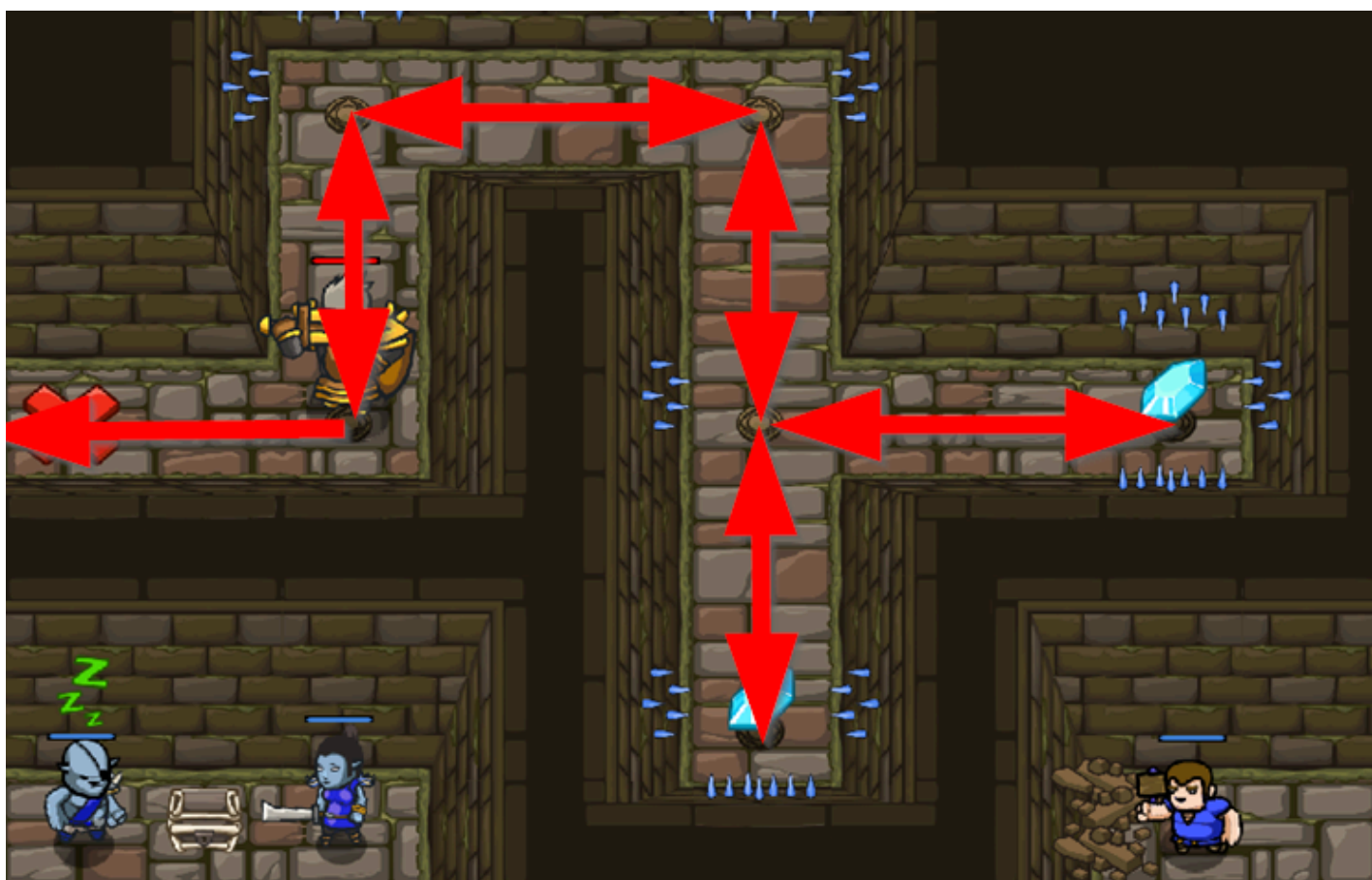
```
// Evite ser visto pelos ogros.  
  
int main() {  
    hero.moveRight();  
    hero.moveLeft();  
    hero.moveRight(2);  
    return 0;  
}
```

#4. Desafio de Conceito. Passos Cuidadosos

Análise do Nível e Soluções

Comandos básicos de movimento.

Introdução



Este é um desafio de conceito.

É hora de verificar o seu conhecimento dos comandos básicos.

Recolha todas as gemas e volte para a saída, que é marcada com a marca X vermelha.

Código padrão

```
int main() {  
    // Este é um desafio de Conceito sobre sintaxe básica.  
    // Recolha todas as gemas e volte para a saída (o X vermelho).  
    // Evite os picos.  
  
    return 0;  
}
```

Visão Geral

Desculpe, não há dicas aqui :-)

É aqui que você usa o que aprendeu nos níveis anteriores.

Se você tiver problemas com essa tarefa, volte e analise os últimos níveis para ter certeza de que os entende!

Passos Cuidadosos Solução

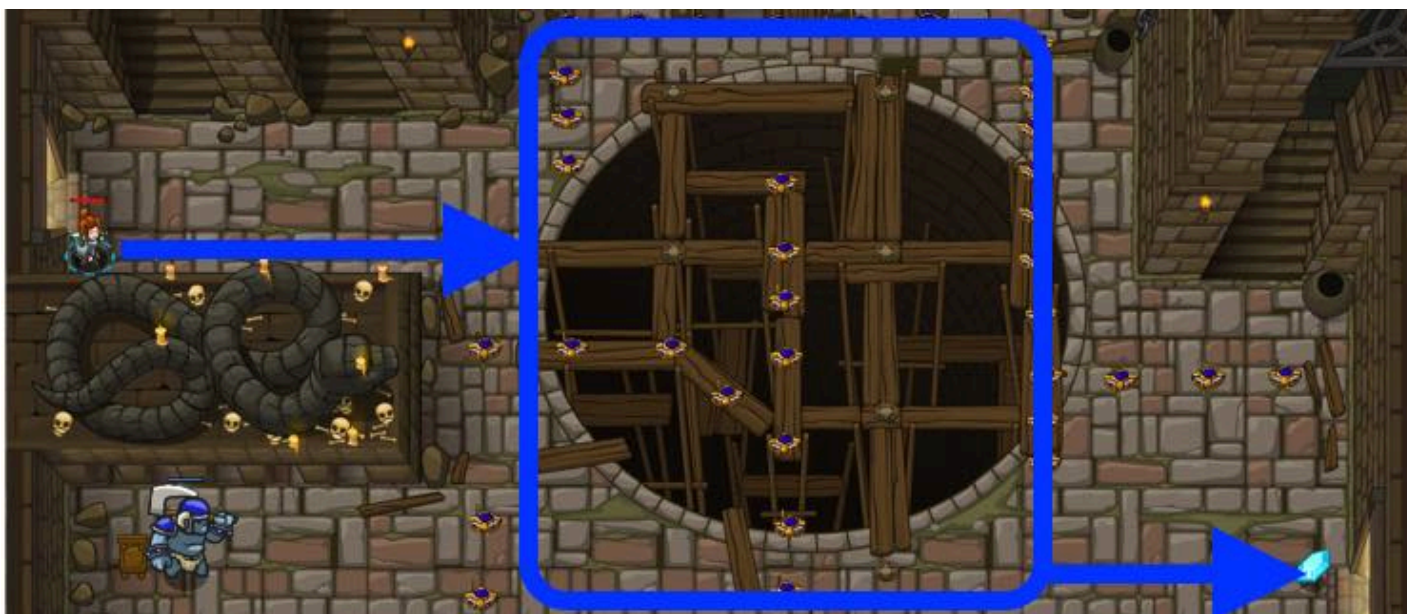
```
int main() {  
    // Este é um desafio de Conceito sobre sintaxe básica.  
    // Recolha todas as gemas e volte para a saída (o X vermelho).  
    // Evite os picos.  
    hero.moveUp();  
    hero.moveRight();  
    hero.moveDown();  
    hero.moveRight();  
    hero.moveLeft();  
    hero.moveDown();  
    hero.moveUp();  
    hero.moveUp();  
    hero.moveLeft();  
    hero.moveDown();  
    hero.moveLeft();  
    hero.moveLeft();  
  
    return 0;  
}
```

#5. Mina Inimiga

Análise do Nível e Soluções

Pise com cuidado, o perigo está no chão.

Introdução



Você pode usar argumentos para simplificar o seu código. Ao invés de:

```
hero.moveRight();  
hero.moveRight();
```

Você pode usar:

```
hero.moveRight(2);
```

Código padrão

```
int main() {  
    // Use argumentos nos comandos para mover-se mais.  
    hero.moveRight(3);  
  
    return 0;  
}
```

Visão Geral

O chão esta cheio de armadilhas de fogo, mas existe um caminho seguro até a gema.

Quando se usa um método como `moveRight()` , você pode fornecer informações extras como "argumentos" ou "parâmetros" para modificar o que ele faz.

Neste caso, ao adicionar um número (argumento) no método `moveRight()` fica assim: `moveRight(3)` . Tendo como resultado a movimentação do herói em 3 espaços para a direita ao invés de 1.

Mina Inimiga Solução

```
int main() {  
    // Use argumentos nos comandos para mover-se mais.  
    hero.moveRight(3);  
    hero.moveUp();  
    hero.moveRight();  
    hero.moveDown(3);  
    hero.moveRight(2);  
    return 0;  
}
```

#5a. Distração Ilusória

Análise do Nível e Soluções

Distraia os guardas e escape.

Introdução

Vá até o **X**, ative a isca (decoy) para distrair os guardas e siga até a gema.

Código padrão

```
int main() {  
    // Ative a isca e pegue a gema. Caso precise, verifique as Sugestões.  
  
    return 0;  
}
```

Visão Geral

Você não conseguirá passar pelos guardas, a não ser que eles estejam distraídos. Felizmente, alguém deixou um isca por perto.

Chegando ao X a isca será ativada.

Dica: Você pode se mover múltiplos espaços ao passar argumentos ao comando move, como 'moveRight(3)'.

Distração Ilusória Solução

```
// Ative a isca e pegue a gema. Caso precise, verifique as Sugestões.  
  
int main() {  
    hero.moveRight();  
    hero.moveDown(2);  
    hero.moveUp(2);  
    hero.moveRight(3);  
    return 0;  
}
```

#5b. Gemas Esquecidas

Análise do Nível e Soluções

Tem gemas espalhadas por toda a Masmorra Kithgard!

Introdução

Colete as gemas usando os comandos de movimento!

```
hero.moveRight();  
hero.moveDown();
```

Código padrão

```
// Pegue as gemas e vá para a saída.  
  
int main() {  
    return 0;  
}
```

Visão Geral

Você irá digitar muitos comandos, tente usar o auto completar para escrever os códigos mais rápido. Ao digitar `g` + Enter o comando `moveRight()` será preenchido automaticamente.

Agora que você aprendeu os movimentos básicos, está na hora de aprender a atacar (`attack`).

Gemas Esquecidas Solução

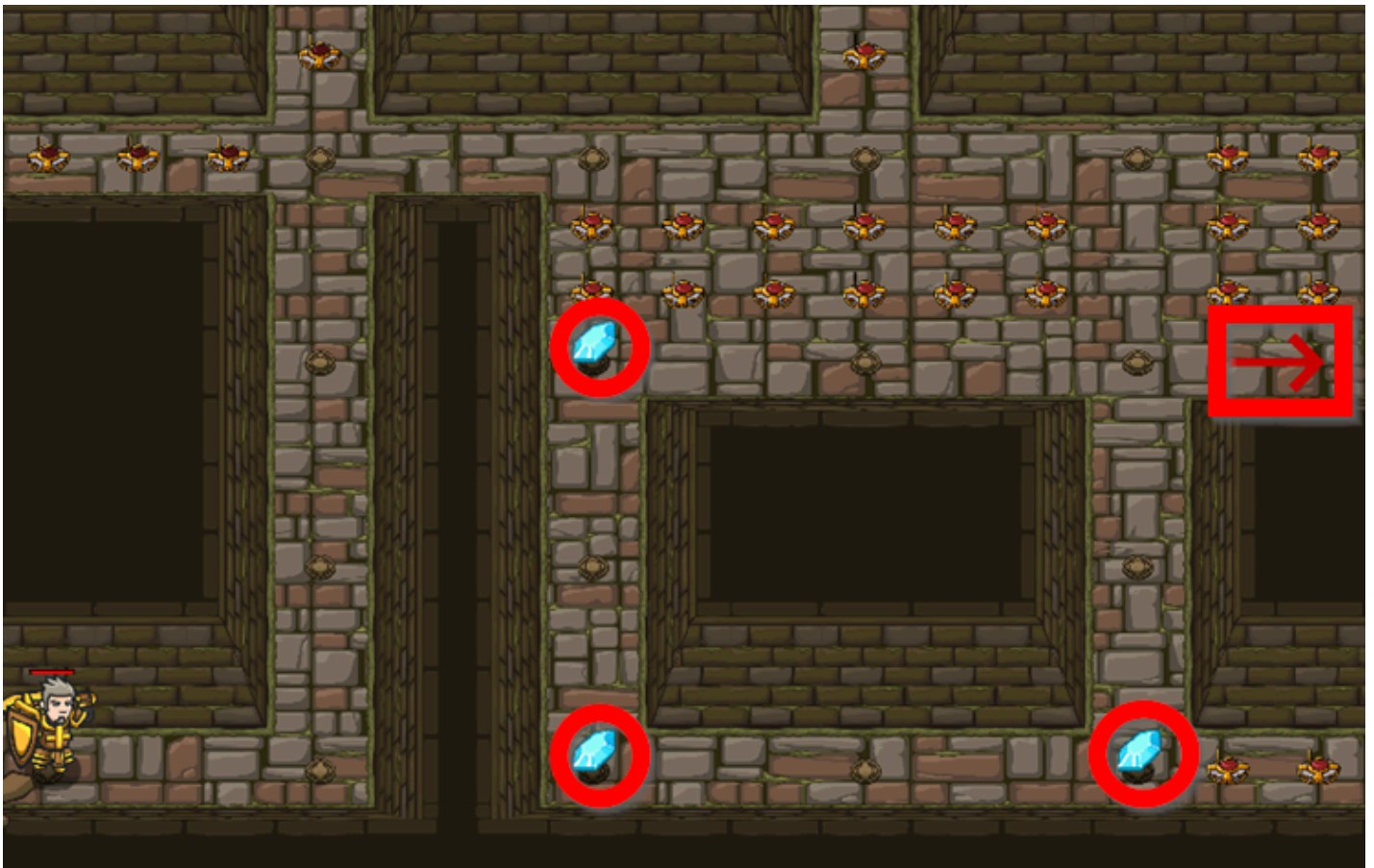
```
// Pegue as gemas e vá para a saída.  
  
int main() {  
    hero.moveRight();  
    hero.moveDown();  
    hero.moveRight(2);  
    hero.moveUp();  
    hero.moveRight();  
    return 0;  
}
```


#6. Desafio de Conceito. Passos Longos

Análise do Nível e Soluções

Usando comandos de movimento com argumentos.

Introdução



Este é um Desafio de Conceito sobre argumentos.

Colete todas as gemas e fuja da masmorra (a seta vermelha marca a saída).

Não use mais que **7 comandos** .

Você precisará usar argumentos com seus comandos de movimento!

Código padrão

```
// Este é um Desafio de Conceito sobre argumentos.  
// Pegue as gemas e vá para a saída.  
// Seu código não deve ter mais de 7 comandos.  
// Use comandos de movimento com argumentos
```

```
int main() {  
    return 0;  
}
```

Passos Longos Solução

```
int main() {  
    // Este é um Desafio de Conceito sobre argumentos.  
    // Pegue as gemas e vá para a saída.  
    // Seu código não deve ter mais de 7 comandos.  
    // Use comandos de movimento com argumentos  
    hero.moveRight(1);  
    hero.moveUp(3);  
    hero.moveRight(2);  
    hero.moveDown(3);  
    hero.moveLeft(1);  
    hero.moveUp(2);  
    hero.moveRight(3);  
  
    return 0;  
}
```

#7. Nomes Próprios

Análise do Nível e Soluções

Aprenda o nome verdadeiro de um inimigo para derrotá-lo

Introdução



Cada ogro precisa ser atacado duas vezes.

Ao chamar o método `attack`, o argumento que será usado é o nome do inimigo ("Nome" / "Name").

Lembre que nomes próprios começam com letras maiúsculas.

Código padrão

```
// Defenda-se contra "Brak" e "Treg"!
// Você precisa atacar os ogros duas vezes.

int main() {
    hero.moveRight();
    hero.attack("Brak");
    hero.attack("Brak");

    return 0;
}
```

Visão Geral

Você deve saber algumas coisas para conseguir terminar este nível:

1. É preciso atacar cada Ogro Munchkin **duas vezes** para derrotá-lo;
2. Digite os nomes corretamente, respeitando as letras maiúsculas e minúsculas!
"Brak" e "Treg" ;
3. Coloque os nomes entre áspas para torná-los strings;
4. Derrote "Brak" , mova-se para a direita e pegue a gema;
5. Depois, derrote "Treg" atacando ele duas vezes;
6. Não tem problema se for derrotado, você sempre pode continuar tentando.

Nomes Próprios Solução

```
// Defenda-se contra "Brak" e "Treg"!
// Você precisa atacar os ogros duas vezes.

int main() {
    hero.moveRight();
    hero.attack("Brak");
    hero.attack("Brak");
    hero.moveRight();
    hero.attack("Treg");
    hero.attack("Treg");
    return 0;
}
```

#7a. Chances Favoráveis

Análise do Nível e Soluções

Dois ogros bloqueiam sua passagem para sair da masmorra.

Código padrão

```
int main() {  
    // Ataque os dois ogros e pegue a gema.  
    hero.moveRight();  
    hero.attack("Krug");  
    hero.attack("Krug");  
  
    return 0;  
}
```

Visão Geral

Lembre-se de atacar os ogros duas vezes, e de colocar seus nomes com as letras maiúsculas entre aspas: "Krug" e "Grump".

Chances Favoráveis Solução

```
int main() {  
    // Ataque os dois ogros e pegue a gema.  
    hero.moveRight();  
    hero.attack("Krug");  
    hero.attack("Krug");  
    hero.moveRight();  
    hero.moveUp();  
    hero.attack("Grump");  
    hero.attack("Grump");  
    hero.moveLeft();  
    hero.moveLeft();  
    return 0;  
}
```

#7b. A Espada Erguida

Análise do Nível e Soluções

Empunhe sua espada para o combate.

Introdução

Ataque cada ogro pelo nome. Lembre-se, é necessario bater duas vezes em cada ogro!

Código padrão

```
// Derrote os ogros.  
// Lembre-se que cada um deles recebem dois golpes.  
  
int main() {  
    return 0;  
}
```

Visão Geral

Tente derrotá-los na ordem que eles chegam, de modo que não possam causar danos extras a você.

A Espada Erguida Solução

Solução Completa

Solução Parcial 1

```
// Derrote os ogros.  
// Lembre-se que cada um deles recebem dois golpes.  
  
int main() {  
    hero.attack("Rig");  
    hero.attack("Rig");  
  
    hero.attack("Gurt");  
    hero.attack("Gurt");  
  
    hero.attack("Ack");  
    hero.attack("Ack");  
    return 0;  
}
```


#8. Desafio de Conceito. Passos Perigosos

Análise do Nível e Soluções

Use strings para derrotar ogros.

Introdução



Este é um Desafio de Conceito: use strings para derrotar ogros pelo nome.

Código padrão

```
// Derrote usando os seus nomes.  
  
int main() {  
    hero.moveRight();  
    // Derrote o primeiro par de ogros.  
  
    hero.moveRight(2);  
    // Derrote o segundo par de ogros.  
  
    return 0;  
}
```

Passos Perigosos Solução

```
// Derrote usando os seus nomes.

int main() {
    hero.moveRight();
    // Derrote o primeiro par de ogros.
    hero.attack("Sog");
    hero.attack("Sog");
    hero.attack("Gos");
    hero.attack("Gos");

    hero.moveRight(2);
    // Derrote o segundo par de ogros.
    hero.attack("Kro");
    hero.attack("Kro");
    hero.attack("Ergo");
    hero.attack("Ergo");

    return 0;
}
```

#9. Desafio Combo. Hora de Dormir

Análise do Nível e Soluções

Use toda a sua proeza de programação para confundir o perigo passado!

Introdução

Este é um nível de desafio COMBO! Complete as metas de nível e pelo menos uma das metas do conceito usando as habilidades de programação que você aprendeu até agora:

- Comandos Básicos
- Argumentos de movimento
- Strings

Se você puder, complete TODOS os objetivos.

Código padrão

```
// Este é um nível de desafio COMBO.  
// Derrote os ogros, colete gemas e fuja para o X vermelho  
// usando strings e argumentos de movimento!  
  
int main() {  
    return 0;  
}
```

Visão Geral

Não ande sobre as pilhas de escombros!

Hora de Dormir Solução

Solução Completa

Solução Parcial 1

Solução Parcial 2

This solution uses all concepts: basic syntax, movement arguments, and strings.

```
int main() {  
    // Este é um nível de desafio COMBO.  
    // Derrote os ogros, colete gemas e fuja para o X vermelho  
    // usando strings e argumentos de movimento!  
    hero.moveRight(2);  
    hero.attack("Ursa");  
    hero.attack("Ursa");  
    hero.moveDown();  
    hero.moveLeft(2);  
    hero.moveUp(2);  
    hero.attack("Rexxar");  
    hero.attack("Rexxar");  
    hero.moveRight();  
    hero.attack("Brack");  
    hero.attack("Brack");  
    hero.moveRight(2);  
  
    return 0;  
}
```

#10. Comentários na Cela

Análise do Nível e Soluções

Você está preso em uma cela com um feiticeiro famoso! Diga a senha para obter a ajuda.

Introdução

No CodeCombat, os comentários dão dicas úteis sobre o que escrever e como estruturar o seu código!

```
// Os comentários em JavaScript começam com //. Leia-os para obter instruções!
```

Código padrão

```
int main() {  
    hero.say("Qual a senha?");  
    // Use a função "say()" para dizer a senha.  
    // A senha é: "Achoo"  
  
    return 0;  
}
```

Visão Geral

Comentários

Os comentários são uma maneira de dois programadores se comunicarem um com o outro. Além disso, são úteis para um único programador lembrar o que estava fazendo caso precise retornar mais tarde!

No CodeCombat, nós adicionamos comentários para ajudá-lo a estruturar seu código e dar dicas vitais. Leia-os para entender qual é o objetivo e o código que você precisa escrever para realizá-lo.

Neste nível, você precisará ler os comentários que nós fornecemos para encontrar a resposta e escapar da cela da prisão!

Comentários na Cella Solução

```
int main() {  
    hero.say("Qual a senha?");  
    // Use a função "say()" para dizer a senha.  
    // A senha é: "Achoo"  
    hero.say("Achoo");  
    hero.moveUp();  
    hero.moveUp();  
  
    return 0;  
}
```

#11. Bibliotecário de Kithgard

Análise do Nível e Soluções

Procure por ajuda de um Bibliotecário aliado!

Introdução



A maioria dos níveis tem dicas que podem ajudá-lo quando você está preso.

Clique em "Próximo" para percorrer todas as Sugestões de um nível.

Código padrão

```
// Você precisa de uma senha para abrir a Porta da Biblioteca!  
// A senha está nas Sugestões!  
// Clique no botão azul acima do editor de código.  
// Não se esqueça que letras maiúsculas e minúsculas geram comandos diferentes.  
  
int main() {  
    hero.moveRight();  
    hero.say("Eu não sei a senha!"); // Δ  
  
    return 0;  
}
```

Visão Geral

Bem vindo as Sugestões!

A maioria dos níveis possuem abas que contém informação extra para ajudar a concluir seus objetivos e aprender novos conceitos.

Por exemplo:

A senha para a Porta da Biblioteca é "Hush" .

Dica: Use o comando `say()` para falar a senha e lembre-se de adicionar aspas na palavra, para transformar a senha em uma string.

Bibliotecário de Kithgard Solução

```
// Você precisa de uma senha para abrir a Porta da Biblioteca!  
// A senha está nas Sugestões!  
// Clique no botão azul acima do editor de código.  
// Não se esqueça que letras maiúsculas e minúsculas geram comandos diferentes.  
  
int main() {  
    hero.moveRight();  
    hero.say("Hush");  
    hero.moveRight();  
    return 0;  
}
```

#12. O Prisioneiro

Análise do Nível e Soluções

Liberte o prisioneiro e você ganhará um aliado.

Código padrão

```
//  
  
int main() {  
    // Liberte Patrick que está atrás da "Weak Door".  
  
    // Mate o guarda, chamado "Two".  
  
    // Pegue o diamante.  
  
    return 0;  
}
```

Visão Geral

You can attack the "Weak Door" by name to free your ally, then help him fight the ogre "Two" .

When you attack something by name, be sure to include the quotes around the name (this is how you specify a String) and be sure to capitalize the names correctly.

Depending on your weapon and your strategy, you may have to attack the ogre more than once.

O Prisioneiro Solução

```
//  
  
int main() {  
    // Liberte Patrick que está atrás da "Weak Door".  
    hero.moveRight();  
    hero.attack("Weak Door");  
    // Mate o guarda, chamado "Two".  
    hero.moveRight(2);  
    hero.moveDown(2);  
    hero.attack("Two");  
    hero.attack("Two");  
    hero.attack("Two");  
    hero.attack("Two");  
    hero.attack("Two");  
    hero.attack("Two");  
    // Pegue o diamante.  
    hero.moveRight();  
    hero.moveDown();  
    return 0;  
}
```

#13. Dança no Fogo

Análise do Nível e Soluções

Poupe digitação (e o seu herói) com loops!

Introdução



Evite as bolas de fogo dançando para a **direita** e **esquerda**.

Use um **loop while-true** para criar um loop infinito:

```
while (true) {  
    hero.moveLeft(); // Isso irá repetir várias vezes.  
}
```

Códigos normalmente executam na ordem em que são escritos. **Loops** te permitem repetir um bloco de código múltiplas vezes sem ter que reescrevê-los.

Código padrão

```
// Códigos normalmente executam na ordem em que são escritos.  
// Loops repetem um bloco de códigos múltiplas vezes.  
  
int main() {  
    while(true) {  
        hero.moveRight();  
        // Adicione um comando moveLeft ao loop aqui  
    }  
  
    return 0;  
}
```

Visão Geral

Como Usar Loops while-true (enquanto-verdadeiro)

Primeiro, nós começamos um loop com a palavra-chave `while` (enquanto). Isso diz ao seu programa que **ENQUANTO** algo for verdadeiro, o **corpo** do loop se repetirá.

Por enquanto, nós queremos que o nosso loop continue executando para sempre, por isso vamos usar o **loop while-true** (loop infinito). Porque verdadeiro é sempre verdadeiro!

Não se preocupe muito com essas coisas de verdadeiro por enquanto. Nós vamos explicar melhor mais adiante. Apenas lembre-se que um **loop while-true** é um loop que não acaba nunca.

É assim que você escreve um **loop while-true**:

```
// Inicie o loop while-true com "while(true) {"  
// Qualquer coisa dentro de { e } é considerada DENTRO do loop.  
while(true) {  
    hero.moveRight();  
    hero.moveLeft();  
}  
  
hero.say("Essa linha não está dentro do loop!");  
  
// Dica: a indentação (espaços nos inícios das linhas) é opcional, no entanto faz com que seu código  
    fique mais fácil de ler!
```

Dica: Você pode por o que quiser dentro de um loop while-true! Mas para este nível, só é necessário repetir dois comandos: `moveRight()` e `moveLeft()`!

Dança no Fogo Solução

```
// Códigos normalmente executam na ordem em que são escritos.  
// Loops repetem um bloco de códigos múltiplas vezes.  
  
int main() {  
    while(true) {  
        hero.moveRight();  
        // Adicione um comando moveLeft ao loop aqui  
        hero.moveLeft();  
    }  
  
    return 0;  
}
```

#14. Labirinto Kithmaze

Análise do Nível e Soluções

Um labirinto construído para confundir viajantes.

Introdução



Verifique a sua identidade!

Você só irá precisar de **um loop while-true** contendo **quatro** comandos neste nível.

Código padrão

```
// A melhor maneira de repetir é usando loops.  
  
int main() {  
    while(true) {  
        // Adicione comandos aqui para repetir.  
        hero.moveRight();  
        hero.moveRight();  
    }  
    return 0;  
}
```


Visão Geral

Loops permitem que você execute o mesmo código repetidas vezes. Você pode passar este nível com apenas quatro comandos usando um **loop while-true** (loop enquanto-verdadeiro).

Garanta que os comandos de movimentos estão **dentro do loop** para que eles se repitam!

Labirinto Kithmaze Solução

```
// A melhor maneira de repetir é usando loops.

int main() {
    while(true) {
        // Adicione comandos aqui para repetir.
        hero.moveRight();
        hero.moveRight();
        hero.moveUp();
        hero.moveUp();
    }
    return 0;
}
```

#15. Continue a Descer

Análise do Nível e Soluções

Use um loop para simplificar o código.

Código padrão

```
int main() {  
    // Atravesse o labirinto em menos de 5 declarações.  
  
    return 0;  
}
```

Visão Geral

Para ganhar o bônus, use um loop com apenas quatro declarações. Linhas em branco e comentários não contam, e múltiplas declarações na mesma linha contam.

Passe o mouse sobre a documentação do loop na área de [Métodos](#) para ver exemplos de sintaxe de loops.

Continue a Descer Solução

```
// Atravesse o labirinto em menos de 5 declarações.  
  
int main() {  
    while(true) {  
        hero.moveRight(2);  
        hero.moveDown();  
    }  
    return 0;  
}
```

#15a. Saindo de Kithmaze

Análise do Nível e Soluções

Se no início você se perder, mude o seu loop para encontrar o caminho.

Código padrão

```
// Use um loop para repetir seus movimentos.  
  
int main() {  
    while(true) {  
        // Escreva seu código aqui para repeti-los.  
        hero.moveRight();  
        hero.moveDown();  
    }  
    return 0;  
}
```

Visão Geral

Conte cuidadosamente quantos movimentos você precisa dentro do seu loop para solucionar o labirinto, e onde você irá repeti-los. Lembre-se, você só deve usar um loop por nível, e garantir que todo o seu código estará dentro dele.

Passe o mouse por cima da documentação do loop para ver um exemplo da sintaxe.

Saindo de Kithmaze Solução

```
// Use um loop para repetir seus movimentos.  
  
int main() {  
    while(true) {  
        // Escreva seu código aqui para repeti-los.  
        hero.moveRight();  
        hero.moveDown();  
        hero.moveRight(2);  
        hero.moveUp();  
    }  
    return 0;  
}
```

#15b. Pedra da Luz

Análise do Nível e Soluções

Esqueletos: criaturas assustadoras ou medrosas?

Código padrão

```
int main() {  
    // Pegue uma Pedra da Luz para que os esqueletos fujam de você por um pequeno intervalo de tempo.  
    return 0;  
}
```

Visão Geral

Esqueletos são muito fortes para se lutar. As Lightstones (Pedras da Luz) podem mantê-los distantes por pouco tempo.

Tudo que você precisa fazer é:

- Coletar uma Pedra de Luz;
- Passar por um esqueleto;
- Repetir.

Pedra da Luz Solução

```
// Pegue uma Pedra da Luz para que os esqueletos fujam de você por um pequeno intervalo de tempo.  
  
int main() {  
    while(true) {  
        hero.moveUp();  
        hero.moveDown();  
        hero.moveRight(2);  
    }  
    return 0;  
}
```

#16. Desafio de Conceito. Loop no Armazém

Análise do Nível e Soluções

Faça um loop na sua rota através do armazém do ogro.

Introdução

Este é um desafio de conceito:

Encontre um padrão de looping para percorrer o armazém, colete todas as gemas, e saia na marca X branca.

Não use mais de 5 declarações.

Código padrão

```
// Colete todas as gemas e fuja na marca X branca.  
// Use não mais que 5 declarações  
  
int main() {  
    return 0;  
}
```

Loop no Armazém Solução

```
int main() {  
    // Colete todas as gemas e fuja na marca X branca.  
    // Use não mais que 5 declarações  
    while(true) {  
        hero.moveUp(2);  
        hero.moveRight(2);  
        hero.moveDown();  
        hero.moveLeft();  
    }  
  
    return 0;  
}
```

#17. Porta Misteriosa

Análise do Nível e Soluções

Atrás de uma porta misteriosa existe um baú cheio de riquezas.

Introdução



Você pode usar **loops while-true** com qualquer método, por exemplo:

```
while(true) {  
    hero.moveRight();  
    hero.attack("Brak");  
}
```

Código padrão

```
// Ataque a porta!  
// Serão necessários muitos ataques, então use um loop "while-true".  
  
int main() {  
    return 0;  
}
```

Visão Geral

Você pode combinar **loops while-true** e o `attack` para causar dano repetidamente. Neste caso, na porta.

```
while(true) {  
    hero.attack("Door");  
}
```

Você pode atacar a porta pelo seu nome, que é "Door".

Com o loop e o ataque, você pode completar este nível usando apenas duas linhas de código.

Porta Misteriosa Solução

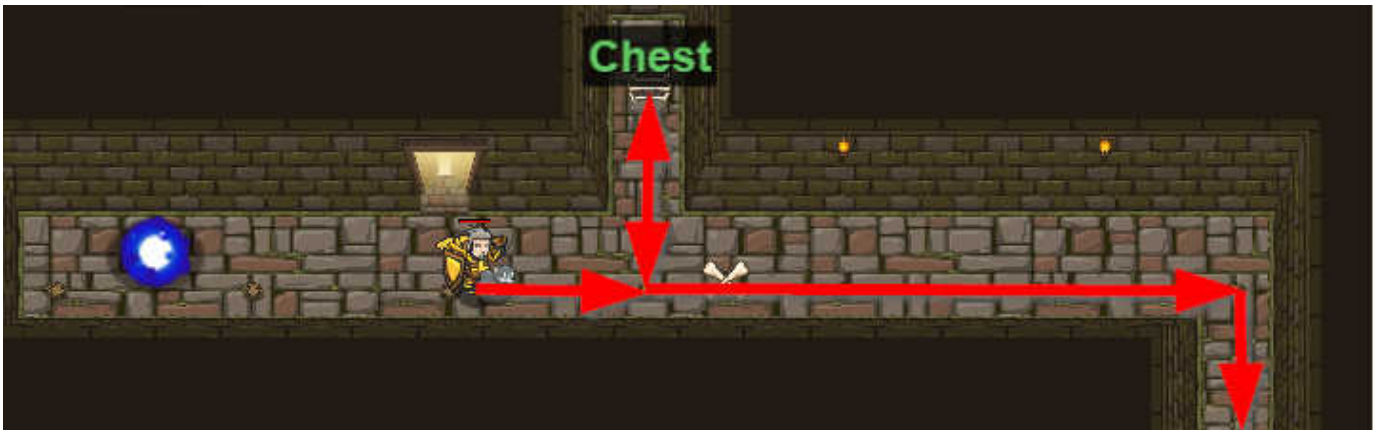
```
// Ataque a porta!  
// Serão necessários muitos ataques, então use um loop "while-true".  
  
int main() {  
    while (true) {  
        hero.attack("Door");  
    }  
    return 0;  
}
```

#18. Bata e Corra

Análise do Nível e Soluções

Fuja do Espírito da Masmorra com a ajuda de uma poção de velocidade.

Introdução



There's danger in the air! You need to escape from the dungeon, but why not check that "Chest" before you go? Use `attack()` to open the "Chest", then use a **while-loop** to escape.

Código padrão

```
int main() {  
    // Você pode escrever códigos antes de um loop.  
    hero.moveRight();  
    // Quebre o "Chest" (baú) antes de usar o loop para escapar do labirinto!  
  
    //  
  
    while(true) {  
        // Mova-se 3 vezes.  
        hero.moveRight(3);  
        //  
    }  
  
    return 0;  
}
```

Visão Geral

Você não é rápido o suficiente para fugir do Espírito sem beber a Poção de Velocidade.

Antes do loop , você irá querer `moveUp()` e acatar o "Chest" , e então `moveDown()` de volta para o corredor principal.

Dentro do loop , você deve adicionar `moveDown()` . Utilize um argumento para `moveDown()` várias vezes sem ter que escrever mais linhas de código.

Não se preocupe, o Espírito é ativado ao se pisar no X, então você terá tempo de pegar a poção!

Bata e Corra Solução

```
int main() {  
    // Você pode escrever códigos antes de um loop.  
    hero.moveRight();  
    // Quebre o "Chest" (baú) antes de usar o loop para escapar do labirinto!  
    hero.moveUp();  
    hero.attack("Chest");  
    //  
    hero.moveDown();  
    while(true) {  
        // Mova-se 3 vezes.  
        hero.moveRight(3);  
        //  
        hero.moveDown(3);  
    }  
    return 0;  
}
```

#19. Armários de Kithgard

Análise do Nível e Soluções

Quem sabe que horrores espreitam nos Armários de Kithgard?

Introdução



Você pode executar qualquer ação antes de um `loop while-true`.

```
hero.moveUp();
while(true) {
    hero.attack("Brak");
}
```

Código padrão

```
// Talvez tenha algo por perto para te ajudar!

int main() {
    // Primeiro, mova-se para o Cupboard (Armário).

    // Em seguida, ataque o "Cupboard" dentro de um loop while-true.

    return 0;
}
```

Visão Geral

Talvez você não seja forte o suficiente para enfrentar os guardas ogros. No armário, você pode encontrar algo útil.

O armário ("Cupboard") está trancado, você terá que atacá-lo repetidamente usando um **loop while-true**.

Armários de Kithgard Solução

```
// Talvez tenha algo por perto para te ajudar!

int main() {
    // Primeiro, mova-se para o Cupboard (Armário).
    hero.moveUp();
    hero.moveRight(2);
    hero.moveDown(2);

    // Em seguida, ataque o "Cupboard" dentro de um loop while-true.
    while (true) {
        hero.attack("Cupboard");
    }
    return 0;
}
```

#19a. Armários de Kithgard A

Análise do Nível e Soluções

Quem sabe que horrores espreitam nos Armários de Kithgard?

Introdução



Você pode executar qualquer ação antes do loop while-true .

```
hero.moveUp();
while(true) {
    hero.attack("Brak");
}
```

Código padrão

```
// Talvez tenha algo por perto para te ajudar!

int main() {
    // Primeiro, mova-se para o Cupboard (Armário).

    // Em seguida, ataque o "Cupboard" dentro de um loop while-true.

    return 0;
}
```

Visão Geral

Talvez você não seja forte o suficiente para os guardas ogros. No armário, você pode encontrar algo útil.

O armário ("Cupboard") está trancado, você terá que atacá-lo repetidamente usando um **loop while-true**.

Armários de Kithgard A Solução

```
// Talvez tenha algo por perto para te ajudar!

int main() {
    // Primeiro, mova-se para o Cupboard (Armário).
    hero.moveDown();
    hero.moveLeft(2);
    hero.moveUp(2);

    // Em seguida, ataque o "Cupboard" dentro de um loop while-true.
    while (true) {
        hero.attack("Cupboard");
    }
    return 0;
}
```

#19b. Armários de Kithgard B

Análise do Nível e Soluções

Quem sabe que horrores se escondem nos armários de Kithgard?

Introdução



Você pode executar qualquer ação antes do `while-true loop`.

```
hero.moveUp();
while(true) {
    hero.attack("Brak");
}
```

Código padrão

```
// Talvez tenha algo por perto para te ajudar!

int main() {
    // Primeiro, mova-se para o Cupboard.

    // Em seguida, ataque o "Cupboard" dentro de um while-true loop (loop while verdadeiro).

    return 0;
}
```

Visão Geral

O primeiro guarda ogre talvez seja muito forte para você enfrentá-lo. Talvez você ache algo que possa te ajudar no "Cupboard" ?

Primeiro, mova-se para perto do "Cupboard" (fique no X). Está trancado, você deve atacá-lo repetidas vezes usando o **while-true loop** para abri-lo.

Armários de Kithgard B Solução

```
// Talvez tenha algo por perto para te ajudar!

int main() {
    // Primeiro, mova-se para o Cupboard.
    hero.moveRight();
    hero.moveDown();
    hero.moveRight();
    hero.moveDown(2);

    // Em seguida, ataque o "Cupboard" dentro de um while-true loop (loop while verdadeiro).
    while (true) {
        hero.attack("Cupboard");
    }
    return 0;
}
```

#20. Inimigo Conhecido

Análise do Nível e Soluções

Use sua primeira variável para conseguir a vitória.

Introdução



The diagram illustrates that the code `self.attack("Kratt")` is functionally identical to `self.attack(enemy)` where `enemy = "Kratt"`. A blue arrow points from the word "Variable" to the `enemy` variable in the second line of code, highlighting its role as a variable.

Declare uma variável assim:

```
auto enemy1 = "Kratt";
```

Quando se usa aspas: "Kratt" , você está fazendo uma **string**.

Quando não usa aspas: `enemy1` , você está se referindo a **variável** `enemy1` .

Código padrão

```
// Você pode usar uma variável como um "crachá de identificação".

int main() {
    auto enemy1 = "Kratt";
    auto enemy2 = "Gert";
    auto enemy3 = "Ursa";

    hero.attack(enemy1);
    hero.attack(enemy1);

    hero.attack(enemy2);

    return 0;
}
```


Visão Geral

Até agora, você estava fazendo três coisas:

1. Chamando **métodos** (como `moveRight()`);
2. Passando **strings** como argumentos para os métodos (citar textos entre aspas, como `"Treg"`);
3. Usando **loops while-true** para repetir os métodos várias vezes.

Agora você está aprendendo a usar **variáveis**: símbolos que representam dados. Os valores das variáveis podem **variar** a medida que você armazena dados nela, por isso o nome variável.

É bem cansativo ter que digitar o nome dos ogros várias vezes. Neste nível você pode utilizar três variáveis para guardar os nomes deles. Dessa forma, quando for atacar você pode usar a variável (`enemy1`) para representar a string que está armazenada nela (`"Kratt"`).

Inimigo Conhecido Solução

```
// Você pode usar uma variável como um "crachá de identificação".
```

```
int main() {  
    auto enemy1 = "Kratt";  
    auto enemy2 = "Gert";  
    auto enemy3 = "Ursa";  
  
    hero.attack(enemy1);  
    hero.attack(enemy1);  
  
    hero.attack(enemy2);  
    hero.attack(enemy2);  
  
    hero.attack(enemy3);  
    hero.attack(enemy3);  
  
    return 0;  
}
```

#21. Mestre dos Nomes

Análise do Nível e Soluções

Use os seus novos poderes de programação para atingir inimigos sem nome.

Introdução

```
enemy1 = self.findNearestEnemy() ★ This sets enemy1 to the nearest enemy.  
self.attack(enemy1) ★  
self.attack(enemy1) ★ These attack the enemy in the variable enemy1.  
  
enemy2 = self.findNearestEnemy() ★ This sets enemy2 to the nearest enemy.  
self.attack(enemy2) ★ This attacks the enemy in the variable enemy2.  
self.attack("enemy2") 🚫 This attacks an enemy with name "enemy2"
```

Variáveis contém informações que podem ser referenciadas depois. Você pode atribuir um novo valor para a variável a qualquer momento.

Use `findNearestEnemy()` para mirar no inimigo mais próximo.

```
auto_closestEnemy = hero.findNearestEnemy();
```

Código padrão

```
// Seu herói não sabe o nome destes inimigos!  
// Use o método findNearestEnemy, concebido pelos óculos.  
  
int main() {  
    //  
    auto enemy1 = hero.findNearestEnemy();  
    //  
    hero.attack(enemy1);  
    hero.attack(enemy1);  
  
    //  
    auto enemy2 = hero.findNearestEnemy();  
    hero.attack(enemy2);  
    hero.attack(enemy2);  
  
    //  
  
    //  
  
    return 0;  
}
```

Visão Geral

Lembre-se do último nível, **variáveis** são símbolos que representam dados.

Agora, ao invés de usar o nome dos inimigos, você pode usar o método `findNearestEnemy()` para armazenar as informações dos ogros nas variáveis.

Quando chamar o método `findNearestEnemy()`, você **deve armazenar o resultado em uma variável**, como `enemy3` (você pode nomeá-la como quiser). A variável irá saber qual **era** o inimigo mais próximo quando o método `findNearestEnemy()` foi chamado, tenha certeza de chamar quando você vir um inimigo nas proximidades.

Lembre-se: quando se usa aspas, como em `"Kratt"`, você está fazendo uma **string**. Quando não usa aspas, como em `enemy1`, você está referenciando a **variável** `enemy1`.

Ogros Munchkins precisam de dois ataques para serem derrotados.

Mestre dos Nomes Solução

```
// Seu herói não sabe o nome destes inimigos!  
// Use o método findNearestEnemy, concebido pelos óculos.
```

```
int main() {  
    //  
    auto enemy1 = hero.findNearestEnemy();  
    //  
    hero.attack(enemy1);  
    hero.attack(enemy1);  
  
    //  
    auto enemy2 = hero.findNearestEnemy();  
    hero.attack(enemy2);  
    hero.attack(enemy2);  
  
    //  
    auto enemy3 = hero.findNearestEnemy();  
    //  
    hero.attack(enemy3);  
    hero.attack(enemy3);  
  
    return 0;  
}
```

#21a. Ataque Duplo

Análise do Nível e Soluções

Use seus óculos para buscar e atacar os ogros.

Código padrão

```
// Crie uma segunda variável e ataque!  
  
int main() {  
    auto enemy1 = hero.findNearestEnemy();  
    hero.attack(enemy1);  
    hero.attack(enemy1);  
  
    return 0;  
}
```

Visão Geral

Como você não sabe o nome dos ogros, pode usar o método `findNearestEnemy` de seus óculos para armazenar referências aos ogros em variáveis.

Quando você chama o método `findNearestEnemy()`, você **deve armazenar o resultado em uma variável**, como `enemy3` (você pode nomeá-la como quiser). A variável irá saber qual **era** o inimigo mais próximo quando o método `findNearestEnemy()` foi chamado. Tenha certeza de chamar quando você vir um inimigo nas proximidades.

Lembre-se: quando se usa aspas, como em `"Kratt"`, você está fazendo uma **string**. Quando não usa aspas, como em `enemy1`, você está referenciando a **variável** `enemy1`.

Ataque Duplo Solução

// Crie uma segunda variável e ataque!

```
int main() {  
    auto enemy1 = hero.findNearestEnemy();  
    hero.attack(enemy1);  
    hero.attack(enemy1);  
  
    enemy1 = hero.findNearestEnemy();  
    hero.attack(enemy1);  
    hero.attack(enemy1);  
  
    hero.moveDown();  
    hero.moveRight();  
    hero.moveRight();  
  
    return 0;  
}
```

#21b. Curta Distância

Análise do Nível e Soluções

Alguns inimigos vão precisar de técnicas de batalha diferentes.

Código padrão

```
int main() {  
    hero.moveRight();  
  
    // Como foi visto no nível anterior:  
    auto enemy1 = hero.findNearestEnemy();  
    // Agora ataque o enemy1.  
  
    return 0;  
}
```

Visão Geral

Assim como no nível anterior, você pode usar o método `findNearestEnemy` para armazenar referências aos ogros em variáveis.

Quando você chama o método `findNearestEnemy`, você **deve armazenar o resultado em uma variável**, como `enemy2` (você pode nomeá-la como quiser). A variável irá saber qual **era** o inimigo mais próximo quando o método `findNearestEnemy` foi chamado, tenha certeza de chamar quando você vir um inimigo nas proximidades.

Lembre-se: quando você usar `findNearestEnemy`, você precisa estar na linha de visão de um inimigo, ou não será capaz de atacá-lo.

Este nível introduz um novo tipo de ogro, o Ogro Atirador (thrower), que causa muito mais dano que os Ogros Munchkins que você já conhece. Existem muitos outros tipos de ogros te aguardando nos próximos níveis.

Curta Distância Solução

```
int main() {  
    hero.moveRight();  
  
    // Como foi visto no nível anterior:  
    auto enemy1 = hero.findNearestEnemy();  
    // Agora ataque o enemy1.  
  
    hero.attack(enemy1);  
    hero.attack(enemy1);  
  
    hero.moveRight();  
  
    enemy1 = hero.findNearestEnemy();  
    hero.attack(enemy1);  
  
    hero.moveRight(2);  
    return 0;  
}
```

#22. Desafio de Conceito. Mestre Dos Nomes de Depurar

Análise do Nível e Soluções

Use seus novos poderes de codificação para atacar inimigos sem nome.

Introdução



Este é um Desafio de Conceito sobre variáveis.

Este é um nível especial de **DEPURAÇÃO** .

O código para o nível é dado a você, mas contém um ou mais erros.

Seu trabalho é descobrir qual é o erro e corrigi-lo!

Código padrão

```
// Há um erro no código abaixo.  
// Tente descobrir o que é, então conserte!  
  
int main() {  
    auto enemy1 = hero.findNearestEnemy();  
    hero.attack(enemy1);  
    hero.attack(enemy1);  
  
    auto enemy2 = hero.findNearestEnemy();  
    hero.attack(enemy2);  
    hero.attack(enemy2);  
  
    auto enemy = hero.findNearestEnemy();  
    hero.attack(enemy3);  
    hero.attack(enemy3);  
    return 0;  
}
```

Visão Geral

Lembre-se do último nível, **variáveis** são símbolos que representam dados.

Agora, ao invés de usar o nome dos inimigos, você pode usar o método `findNearestEnemy()` para armazenar as informações dos ogros nas variáveis.

Quando chamar o método `findNearestEnemy()`, você **deve armazenar o resultado em uma variável**, como `enemy3` (você pode nomeá-la como quiser). A variável irá saber qual **era** o inimigo mais próximo quando o método `findNearestEnemy()` foi chamado, tenha certeza de chamar quando você vir um inimigo nas proximidades.

Lembre-se: quando se usa aspas, como em `"Kratt"`, você está fazendo uma **string**. Quando não usa aspas, como em `enemy1`, você está referenciando a **variável** `enemy1`.

Ogros Munchkins precisam de dois ataques para serem derrotados.

Mestre Dos Nomes de Depurar Solução

```
// Há um erro no código abaixo.  
// Tente descobrir o que é, então conserte!
```

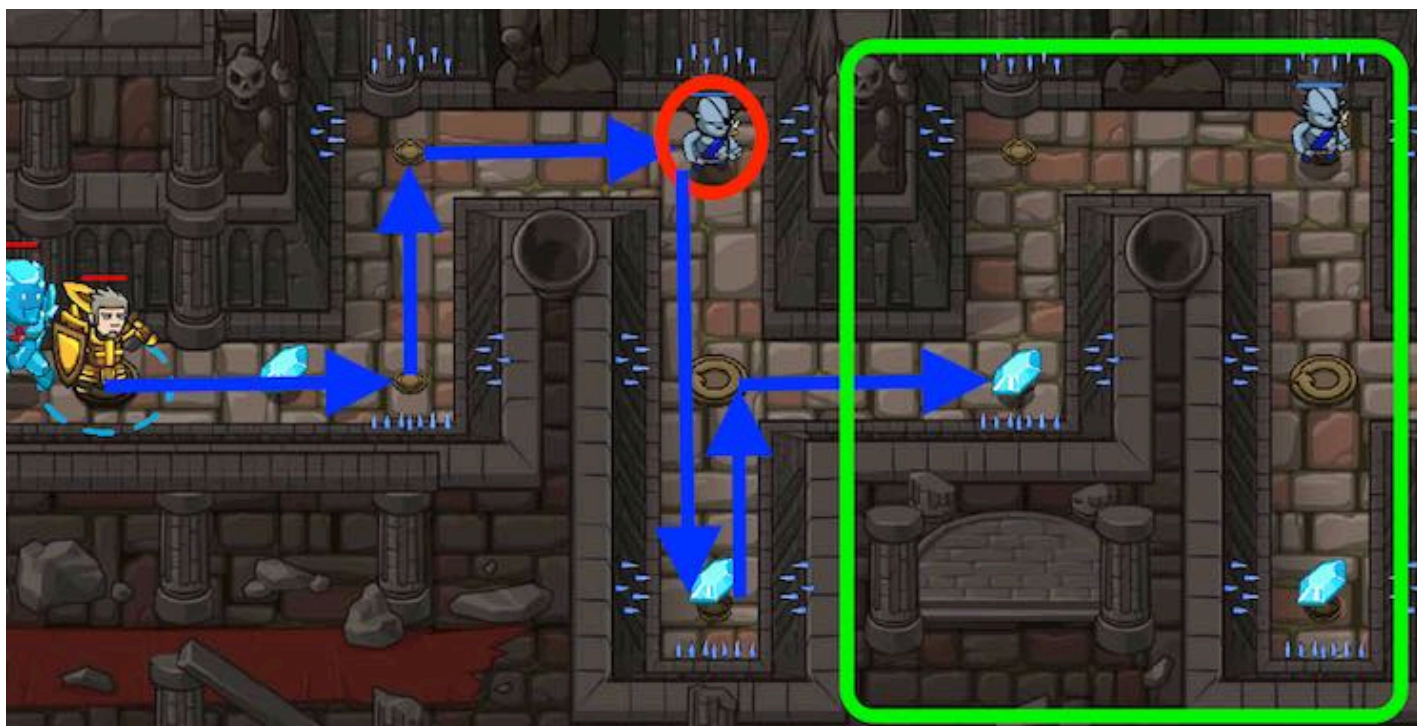
```
int main() {  
    auto enemy1 = hero.findNearestEnemy();  
    hero.attack(enemy1);  
    hero.attack(enemy1);  
  
    auto enemy2 = hero.findNearestEnemy();  
    hero.attack(enemy2);  
    hero.attack(enemy2);  
  
    auto enemy3 = hero.findNearestEnemy();  
    hero.attack(enemy3);  
    hero.attack(enemy3);  
    return 0;  
}
```

#23. O Labirinto Final

Análise do Nível e Soluções

Para escapar você precisa atravessar o labirinto do velho Kithman.

Introdução



Lembre-se de chamar a função `findNearestEnemy()` apenas quando o inimigo estiver no campo de visão.

Código padrão

```
// Use um loop while-true para mover-se e atacar.  
  
int main() {  
    while(true) {  
  
        }  
        return 0;  
    }  
}
```

Visão Geral

Este nível combina **loops while-true** e **variáveis** para resolver o labirinto e atacar os inimigos.

Agora você sabe porquê as variáveis são necessárias, pois pode mudar os dados guardados nelas. Dentro do seu loop while-true (enquanto-verdadeiro), se você definir uma variável `enemy`, ela irá se referir a cada um dos três Ogros Munchkins que aparecem neste nível a medida que o loop se repete. Legal, né?

Preste atenção onde o seu loop while-true deve repetir, para que você não se mova mais do que o necessário.

O Labirinto Final Solução

```
// Use um loop while-true para mover-se e atacar.

int main() {
    while (true) {
        hero.moveRight();
        hero.moveUp();
        auto enemy = hero.findNearestEnemy();
        hero.attack(enemy);
        hero.attack(enemy);
        hero.moveRight();
        hero.moveDown(2);
        hero.moveUp();
    }
    return 0;
}
```

#24. Desafio Combo. Lugar Seguro

Análise do Nível e Soluções

Use argumentos, loops e variáveis para derrotar os ogros

Introdução

Este é um desafio COMBO!

Você precisa derrotar os ogros usando pelo menos uma das habilidades de programação que você aprendeu até agora:

- Variáveis
- While Loops
- Argumentos

Se você puder, complete TODOS os objetivos!

Código padrão

```
// Mova-se para a sala do tesouro e derrote todos os ogros.  
  
int main() {  
    return 0;  
}
```

Lugar Seguro Solução

Solução Completa

Solução Parcial 1

Solução Parcial 2

Solução Parcial 3

This solution uses all concepts: arguments, while loops, and variables.

```
int main() {  
    // Mova-se para a sala do tesouro e derrote todos os ogros.  
    hero.moveUp(4);  
    hero.moveRight(4);  
    hero.moveDown(3);  
    hero.moveLeft(2);  
    while(true) {  
        auto enemy = hero.findNearestEnemy();  
        hero.attack(enemy);  
        hero.attack(enemy);  
    }  
  
    return 0;  
}
```

#25. Portões de Kithgard

Análise do Nível e Soluções

Escape da Masmorra Kithgard e não deixe que os guardiões o peguem.

Introdução



O método `buildXY("fence", x, y)` permite que você construa uma cerca em um determinado local. Por exemplo, para as coordenadas $x = 40$ e $y = 20$:

```
hero.buildXY("fence", 40, 20);
```

Passe o mouse sobre o mapa para obter os valores das coordenadas de onde você quer construir a cerca e substitua esses números nos argumentos X e Y da função `buildXY`.

Código padrão


```
// Construa 3 cercas para manter os ogros isolados!

int main() {
    hero.moveDown();
    hero.buildXY("fence", 36, 34);

    return 0;
}
```

Visão Geral

Quando você usa um martelo de construtor, ao invés de uma espada, você recebe o método `buildXY`. O método `buildXY` recebe três argumentos ao invés de um: `buildType`, `x`, e `y`. Isso te permite escolher o que construir e onde construir.

- `buildType`: utiliza as strings `"fence"`, para construir cercas, e `"fire-trap"`, para construir armadilhas de fogo;
- `x`: é a posição horizontal de onde irá construir;
- `y`: é a posição vertical de onde irá construir.

Você pode **passar o mouse sobre o mapa** para descobrir as coordenadas `x` e `y`. Ambos os valores estão em metros.

Para construir uma cerca (`"fence"`) nas coordenadas `x = 40` e `y = 20`:

```
hero.buildXY("fence", 40, 20);
```

Este nível é **muito mais fácil de ser concluído construindo "fence"** do que `"fire-trap"`. É praticamente impossível utilizar armadilhas de fogo para derrotar os ogros. Se você quiser tentar, fique a vontade, mas nós levamos quinze minutos para descobrir. E olhe que nós construímos o nível.

Portões de Kithgard Solução

```
// Construa 3 cercas para manter os ogros isolados!

int main() {
    hero.moveDown();
    hero.buildXY("fence", 36, 34);
    hero.buildXY("fence", 36, 30);
    hero.buildXY("fence", 36, 26);
    hero.moveRight(3);
    return 0;
}
```


#26. Wakka Maul

Análise do Nível e Soluções

![[Nov17 wakka maul]](/file/db/level/5630eab0c0fcbd86057cc2f8/NOV17-Wakka Maul.png) Batalhe com os seus colegas enquanto coleta gemas! Use as suas habilidades e criatividade como programador para adquirir vantagem sobre eles.

Introdução



Ande sobre as gemas para coletá-las.

Use o método `say()` com o tipo das unidades para convocá-las.

E use `attack()` para destruir as portas e libertar seus aliados.

Código padrão

```
// Bem vindo a Wakka Maul! Prepare-se para o combate!
// Arrisque-se pelo labirinto e pegue algumas gemas para a guerra.
// Destrua as portas para libertar aliados (ou inimigos).
// Por exemplo, para atacar a porta "g" utilize:
//hero.attack("g")
// Se você possuir ouro suficiente, você pode chamar por ajuda dizendo o nome da unidade que você
// gostaria de convocar!
//hero.say("soldier") para convocar um Soldado ao custo de 20 de ouro!
//hero.say("archer") para convocar um Arqueiro ao custo de 25 de ouro!

int main() {
    hero.moveDown();
    hero.moveRight();
    hero.attack("g");
    hero.say("soldier");

    return 0;
}
```

Visão Geral

Wakka Maul

Enfrente seus amigos, colegas de trabalho e de classe nessa batalha pela Masmorra Kithgard!

Liberte os seus aliados, chame mais unidades e desvie dos ataques inimigos.

As portas estão nomeadas como: "a" , "b" , "c" , "d" , "e" , "f" , "g" , "h" , "i" , "j" . Use essas "strings" para atacar uma porta específica de seu interesse.

O lado human (humano) pode chamar soldier e archer (soldados e arqueiros) enquanto o lado ogro pode chamar scout e thrower (batedores e atiradores). Ambos os lados precisam dizer o nome da unidade e ter gemas suficientes para convocá-las. Para convocar as unidades você deverá dizer seus nomes:

```
// Se estiver no lado dos Humanos:
hero.say("soldier"); // Para convocar um soldado por 20 de ouro.
hero.say("archer"); // Para convocar um arqueiro por 25 de ouro.

// Se estiver no lado dos Ogros:
hero.say("scout"); // Para convocar um batedor por 18 de ouro.
hero.say("thrower"); // Para convocar 2 atiradores por 9 de ouro cada.
```

Wakka Maul Solução

```
// Bem vindo a Wakka Maul! Prepare-se para o combate!
// Arrisque-se pelo labirinto e pegue algumas gemas para a guerra.
// Destrua as portas para libertar aliados (ou inimigos).
// Por exemplo, para atacar a porta "g" utilize:
//hero.attack("g")
// Se você possuir ouro suficiente, você pode chamar por ajuda dizendo o nome da unidade que você
// gostaria de convocar!
//hero.say("soldier") para convocar um Soldado ao custo de 20 de ouro!
//hero.say("archer") para convocar um Arqueiro ao custo de 25 de ouro!

int main() {
    hero.moveDown();
    hero.moveRight();
    hero.attack("g");
    hero.moveRight(4);
    hero.moveUp();
    hero.attack("h");
    hero.attack("i");
    hero.moveUp(2);
    while (true) {
        hero.say("archer");
    }
    return 0;
}
```
